

What Language Models Miss in the Option Spaces of the Programs They Call

Anonymous Author(s)

Submitted for double-anonymous review

Abstract—Command line programs expose their behaviour through options, and a single program can document hundreds of them. Language models now write and execute these commands at scale, yet it is unclear whether they use the breadth of options a program offers or only the few forms common in training data. To find out, we analyse 31,145 invocations of 403 programs covering GNU coreutils, Git, and CI tooling (9,604 documented options in total), drawn from model generated scripts, open source repositories, and the Linux kernel build system.

Our results show that about nine in ten options are never used by anyone. Models reach 5.4% of the GNU option space, nearly identical to the human average, while the kernel expert reaches 7.03%. Moreover, where both use the same option, they rank and spell it the same. What separates the expert, however, is selecting the right option for a concrete task, because a requirement explicitly names it. We also find that agents that can execute commands and observe their output do not escape this pattern. In a study of two agent systems, we measure reach below 1.3% of the documented space, showing that seeing what a command did gives no signal about the options that were never tried.

Taken together, these findings indicate that models are reliable interpolators of their training distribution, but not independent explorers of the option space of the programs they generate. Because the options a model never selects are also the ones that are rare in training text, scaling alone is unlikely to close the gap. Instead, exposing a program’s option set to the model at generation time appears to be the most direct path forward.

Index Terms—option spaces, configuration spaces, large language models, command line programs, empirical software engineering, software configurability, program configuration

I. INTRODUCTION

Almost every software system is configurable. Rather than ship a single fixed behaviour, programs expose choices, such as whether to show hidden entries, sort output by size, or compress data as it is written. The variability and software product line communities have long modelled such choices as *features* whose combinations define a configuration space [1], [2], [3]. Some features are bound before a program runs, but command-line options are the typical runtime case, and a distinctive one, because developers commit them to their source code artifacts. Concretely, the options, or flags, a program accepts, such as `-l` to list files in long form or `-r` to act recursively in an `ls` program, are written into shell scripts, Dockerfiles, and continuous integration (CI) workflows. Then, a single program can document hundreds of such options [4], more than any user holds in mind, and far more once the combinations and dependencies among them are counted.

For most of the history of shell scripts, container files, and CI workflows, the program invocations they carry were written

Listing 1. Prompted to archive a tree, the model uses 3 of `tar`’s 200+ options.

```
# Prompt: "create a release archive of the source tree."
tar -czf release.tar.gz src/

# When asked how it chose, the model replied:
# "I used the standard form from my training data."
# I did not consult tar's option list."
```

by people. That is changing. Large language models now write a growing share of these artifacts [5], [6], and developers accept the generated suggestions at a pace that leaves little room for review [7], [8]. In each invocation the model decides not only which program to call but also which options to pass, and it is this second decision that determines what the command actually does. We gave one such model (Claude Opus) a routine task, namely to “*create a release archive of the source tree*”. It answered in one line, as shown in Listing 1.

The invocation is valid and runs without error, which is what makes it telling. The model wrote it from memory, never invoking `tar --help` to see what the program documented, and its own explanation says as much. Yet the same binary has more than 200 other options, including flags to keep the archive within one file system (`--one-file-system`) and to verify it after writing (`--verify`). None were consulted, even though adding `--sort=name` and `--mtime` would have produced a byte-for-byte reproducible archive [9], a strictly stronger result than what the prompt required. Reaching these options is, moreover, only the first difficulty. The right choice often depends on other options and on the neighbouring commands a program is combined with, neither of which even the option list by `--help` makes visible (Section II). This raises a concern that goes beyond one prompt. Because a model generates from a prior fixed at training time, the forms it produces should track what was frequent in its training text rather than what a task requires, a selection pressure already documented for insecure patterns in assistant-generated code [10]. If that holds, then as models author more of our scripts, container files, and CI workflows, and increasingly execute them directly through agent shells (Section II), any narrowing they carry would propagate at the scale of model adoption. Whether models actually narrow the configuration space, and how their reach compares to that of human authors, is an empirical question rather than an assumption, and it is the question this paper sets out to answer.

This paper asks three questions. *How much of a program’s*

documented option set is exercised in practice? Does a model reach the same portion as a human, or a different one? And when both cohorts use the same option, do they agree on how they use it? We study these questions across three datasets spanning the GNU command line programs, four families of CI tooling, and the full Git subcommand suite, together covering 403 programs and more than 9,600 documented options. For each program we recover its full documented option set from its own `--help` output. We then assemble a human cohort and a model cohort, extract the options each actually uses, and measure how far each reaches into the documented space of programs' options.

The findings converge across all three domains. Roughly nine in ten documented program options are never used by any caller. The model cohort *reaches* almost exactly as far as the average human, and only an expert reference drawn from the Linux kernel reaches notably wider. Where a model and a human use the same option, they *rank* it alike and *spell* it in the same form. The two cohorts part company not in how they use shared options but in which options each selects. Options used only by the expert tend to be the ones a concrete requirement demands, while both the model and the average human settle on the same familiar portion of the option space. A study of two AI agent systems *that execute* commands shows the same pattern, with reach narrowing further to under 1.3%.

In summary, this paper makes the following contributions:

- A ground truth of the documented option sets of 403 command line programs, serving as a baseline for measuring how callers use these programs.
- A measurement framework with four metrics, namely *reach*, *concentration*, *partition*, and *style*, for comparing how different cohorts of callers use that option space.
- Three curated datasets covering GNU coreutils, CI tooling, and Git (403 units, 9,604 options, ~31,000 invocations), each split into a human and a model cohort, with an expert reference for the GNU programs.
- A study of two agent systems showing that executing commands and observing output does not widen *reach*.
- A replication package containing all scripts, corpora, ground truth files, and the analysis notebook¹.

Section II motivates the study and frames the research questions. Section III describes the datasets, extraction pipeline, and metrics. Section IV reports the findings. Section V discusses the agent study and its implications. Sections VI and VII address threats to validity and related work. Finally, Section VIII concludes the paper and discusses future work.

II. BACKGROUND AND MOTIVATION

A. Options as the Configuration Surface of Programs

A command-line program exposes runtime variability through *options*, that is, flags prefixed with one or two dashes that a caller appends to the program name. For example, `rm -f` suppresses confirmation prompts and `grep -r` searches

recursively. The complete set of options that a program documents forms what we call its *documented option set*, and this set tends to be large. A single program can document hundreds of options, far more than any human can keep in mind [4]. In the variability and software product line literature, such choices are modelled as *features* whose valid *combinations*, together with the constraints among them, define a program's *configuration space* [1], [2], [3]. The documented option set is therefore the flat list of everything a program offers, whereas the configuration space is the broader, constrained space of their valid combinations. Every program ships a *default behaviour*, meaning what it does when no option is supplied at all, and it is the options a caller adds that determine what a program invocation actually does beyond that default. Because developers commit different configurations of these programs (*i.e.*, commands) into shell scripts, container files, and CI workflows [11], the choices are not transient. They persist in source control and run again on every execution.

Knowing the full set of documented options, however, is only the first step toward configuring a program well. Options are not independent, and three difficulties remain even once the list is in view. First, options carry interaction constraints that are mostly not stated in the option list. Research on software configurability has shown that such implicit constraints must typically be recovered through dedicated analysis rather than read off the documentation [12], [13]. Secondly, the relevant configuration often spans several programs in a pipeline, so the correct option for one program depends on what its neighbour program emits. Thirdly, whether the configuration space is ever in view at all differs by caller. A developer can pause and take the time to read a manual page or invoke the help output, whereas a model emits each invocation, often quicker, from its training prior, with no equivalent consultation step unless the options and their constraints are placed before it.

Moreover, getting configuration wrong is a well documented source of failure in software engineering. Empirical studies report a substantial fraction of production incidents to misconfiguration rather than to code defects [14], [15], and surveys describe configuration as an engineering concern in its own right [16], [3]. On the command line the consequences are concrete and familiar. A missing `--sort` flag leaves a build artifact non-reproducible, and omitting a `--timeout` in a CI step means a hanging test can block an entire pipeline indefinitely. The options that would avoid such outcomes exist and are documented, yet the open question is whether the callers who write these commands actually reach for them.

B. Who Writes, and Now Runs, These Invocations

Historically, the human callers who wrote these invocations in project files could consult a manual page or invoke the help output before choosing an option. That consultation step is disappearing. Large language models now author a growing share of the shell scripts, container files, and CI workflows that carry these calls, and developers accept the generated suggestions with limited review [5], [6], [7], [8]. Currently,

¹Anonymised replication package, <https://github.com/anonconfs/llms-miss>.

in each generated invocation the model selects which options to pass without first inspecting the program’s full option set.

A second shift sharpens the question further. Beyond writing static artifacts, models increasingly run commands directly. Recent AI agent systems expose a shell to the model as a built in action, so that the model emits a raw program invocation that is executed in a sandbox and its output is fed back as an observation [17], [18], [19]. This capability is now shipped in widely used AI assistants [20]. In that setting the options a model selects are not merely written down for later review but executed immediately, so the breadth of options it does or does not reach for directly determines what the system actually does. In both cases, whether writing code or executing it, the choice of which options to supply to a program now passes through a model instead of a person, and it is this choice that determines what the command actually does.

C. Why Compare Models Against Humans

These shifts make the question of how models configure the programs they call an empirical one rather than a matter of opinion. A model generates each invocation from a distribution fixed at training time, so the commands (*i.e.*, programs with specific options) it produces tend to reflect what appeared frequently in its training text. We hypothesise that this holds even when the task at hand would benefit from less common options that the model has rarely or never seen. The same selection pressure has already been documented for insecure coding patterns reproduced from common examples [10] and for plausible but non-existent package names [21]. If models inherit only a narrow band of habitual usage, then as they author and execute more of our scripts, that narrowing would propagate at the scale of model adoption.

Whether this actually happens cannot be read off the model in isolation. Any single human caller is unlikely to use an entire option space [4], so observing that a model uses few options would be uninformative without knowing how many a human uses on the same program. What matters is how a model’s reach compares to a sensible baseline, and humans are the natural one. People have written command line invocations for decades, and the programs, their manuals, and conventions in question were built by and for human callers. This makes human usage the appropriate baseline against which a model’s operational reach can be meaningfully evaluated. Comparing the two groups of callers on the same programs separates a limitation specific to models from one shared by anyone who configures these programs, and it lets us ask the sharper question of *where* within the space the two part company.

This comparison is also complementary to how generated code is usually reviewed. Existing benchmarks ask whether generated code passes tests [22] or resolves real repository issues [23], *i.e.*, whether it is functionally correct. The option surface is orthogonal to that question. An invocation can pass every test while still omitting the options that make it robust, reproducible, or composable, and such omissions are invisible to a correctness check. Measuring which options

models actually exercise, on the same programs as humans is therefore the gap this study sets out to fill.

III. STUDY DESIGN

We formulate three research questions and define the datasets, extraction methods, and metrics used to answer them.

A. Goal and Research Questions

Following the Goal-Question-Metric template [24], our goal is to *analyse* command line invocations generated by large language models *for the purpose of* measuring how far models reach into the documented option sets of the programs they call, using human authored invocations as the reference baseline. The comparison is made *with respect to* the breadth and form of the options each cohort exercises, from the *viewpoint of* a software engineer who needs to know whether a generated command is safe and robust, *in the context of* GNU programs, Git subcommands, and CI tooling. From this goal we derive three research questions.

- RQ1 (**Saturation**). **How much of a program’s documented option set is exercised in practice?**
- RQ2 (**Divergence**). **How does the portion of the documented option set reached by language models compare with that reached by humans?**
- RQ3 (**Style**). **When both cohorts use the same option, do they agree on how they use it?**

More specifically, RQ1 characterises the documented option set itself, including how concentrated the usage that does occur is. RQ2 locates each cohort within it, with an expert human reference as the upper bound. RQ3 then asks whether agreement on *which* options also implies agreement on *how* they are used, looking at relative popularity, spelling, and configuration depth.

B. Definitions

We define four terms used throughout this study.

A *unit* is the smallest entity, often referred to as a program or tool, that owns an option set, *i.e.*, configuration space. For the GNU programs a unit is a single binary (*e.g.*, `tar`). For Git a unit is a subcommand (*e.g.*, `git commit`). For the continuous integration tools a unit is a program together with its subcommand, where both the program and the subcommand vary (*e.g.*, `docker build` versus `npm install`).

An *invocation* is a single parameterised call, *i.e.*, given options, to one unit. One artifact, such as a shell script, a CI workflow, or a Makefile, typically contains many invocations, so the invocation, not the file, is our primary unit of analysis.

A *caller* is the author that issues an invocation, *i.e.*, a person writing a shell script or a language model generating one. The caller is the entity whose choice of options we measure.

A *cohort* is a group of callers that share a provenance. We study a human cohort and a language model cohort. Within the human cohort, we mark a subset of expert callers from the Linux kernel that we use in places as a high-skill reference.

C. Datasets

To answer our research questions, we apply the same study methodology to three datasets that differ in domain, scale, and the kind of artifact that carries the invocations (Table I).

Dataset A: GNU and Related Programs: This dataset covers 118 command line programs from GNU coreutils and commonly co-installed system packages, such as `tar`, `grep`, `find`, `gawk`, `sed`, and `diff`, among others, collectively documenting 2,716 options. We assemble three cohorts. The *general human* cohort consists of 293 shell scripts mined from 26 highly starred open source repositories on GitHub, such as `systemd`, `curl`, Docker library images, `ohmyzsh`, and `pyenv`, among others, all committed before 2020. The *model* cohort consists of 298 scripts found in public repositories generated by ChatGPT, Claude, or GPT. In this dataset, we add an *expert* cohort consisting of the 1,081 build and automation scripts embedded in the Linux kernel source tree. The kernel build system is peer reviewed, performance critical, and covers an unusually wide range of system utilities, which makes it a natural reference for expert level configuration. In total the three cohorts yield approximately 28,900 program invocations of the 118 target units (*i.e.*, programs).

Dataset B: CI Tooling: This dataset targets the tool families that dominate modern CI pipelines, a practice now standard in open source development [11], namely Docker (with Docker Compose), `npm`, and `pip`. Together these expose 135 program and subcommand units and 1,864 documented options. The *human* cohort contains 210 GitHub Actions and CI workflow files drawn from 28 major open source projects, such as TensorFlow, PyTorch, Node.js, Django, and Angular, among others, all committed before late 2022. The *model* cohort contains 210 AI generated CI pipelines drawn from 180 public repositories with declared model provenance. This is our smallest corpus, approximately 2.6 megabytes of YAML with 282 invocations, and we flag it as such when interpreting results. We include it because CI pipelines run automatically on every code change, so a single missing option, such as a cache control or a timeout, can directly cause build failures, security exposures, or wasted compute.

Dataset C: Git: This dataset covers the full suite of 150 Git subcommands, which expose 5,024 documented options. The *human* cohort comprises 276 shell scripts from open source projects. The *model* cohort comprises 468 generated files. Git complements the GNU programs because its subcommand architecture creates a deeper, more layered configuration environment, in which a caller must navigate both the choice of subcommand and the options within it.

D. Ground Truth Recovery

For every unit in each dataset we recover its documented option space directly from the installed binary (*i.e.*, GNU coreutils 9.10, Git 2.54, Docker 29.4, `npm` 10.9, and `pip` 26.0) by invoking its help interface (`--help` or the appropriate subcommand `help`). We parse the resulting text with a regular expression based extractor that identifies short options (`-X`) and long options (`--word`) and records each as a distinct

TABLE I
SUMMARY OF THE THREE DATASETS, GNU, CI, AND GIT.

	A. GNU	B. CI	C. Git
Units (tools or subcommands)	118	135	150
Documented options	2,716	1,864	5,024
Scripts, human	293	210	276
Scripts, model	298	210	468
Scripts, expert (kernel)	1,081	—	—
Corpus size (MB)	8.8	2.6	10.6
Invocations, human	4,837	96	564
Invocations, model	8,795	186	1,268
Invocations, expert	15,271	—	—

entry. The output is one ground truth file per unit, listing every option, and a summary that counts short, long, and total options. Applying this procedure across the three datasets, we recovered 9,604 documented options over 403 units.

Two program families required extensions to this basic protocol. For Git, no single help surface is complete. The minimal `-h` output of several subcommands is a bare usage stub, some documented synonyms such as the `--staged` of `git diff` appear only in manual page prose, and bracketed forms such as `--[no-]verify` compress two options into one token. We therefore take the union of three surfaces per subcommand, namely the `-h` output, the option definition lines of the manual page, and the token list of Git’s own completion helper, expanding bracketed forms into both spellings. For `find` and `stty`, which predate the GNU option conventions, we additionally extract the single dash word options (*e.g.*, `-name`, `-type`) that the standard patterns miss.

We recover the ground truth from the installed binary rather than from online documentation, because the help output is the same place a caller consults on the system where the program runs and it fixes the option set to a known version. Then, online manuals are not uniformly available across all 403 units and may describe a different version than the one installed.

E. Invocation Extraction

From each corpus artifact we extract every invocation of a target unit, *e.g.*, from a shell script we capture `tar -czf out.tgz src/`, an invocation of the unit `tar`. This extraction proceeds in two steps.

Step 1 (locating invocations): For shell scripts (*i.e.*, Datasets A and C) we parse the command stream to locate occurrences of the target program names. We capture the program name, the subcommand if applicable, and all tokens that follow up to the next command separator, such as pipe, semicolon, ampersand, or line termination. For YAML based CI workflows (*i.e.*, Dataset B) we parse the `run:` blocks to extract the embedded shell commands and apply the same token extraction. For the kernel corpus we apply identical parsing to the Makefile recipes and embedded shell fragments.

Step 2 (normalising options): Each captured token is classified as either an option (*i.e.*, it begins with a dash and matches an entry in the unit’s ground truth) or a positional

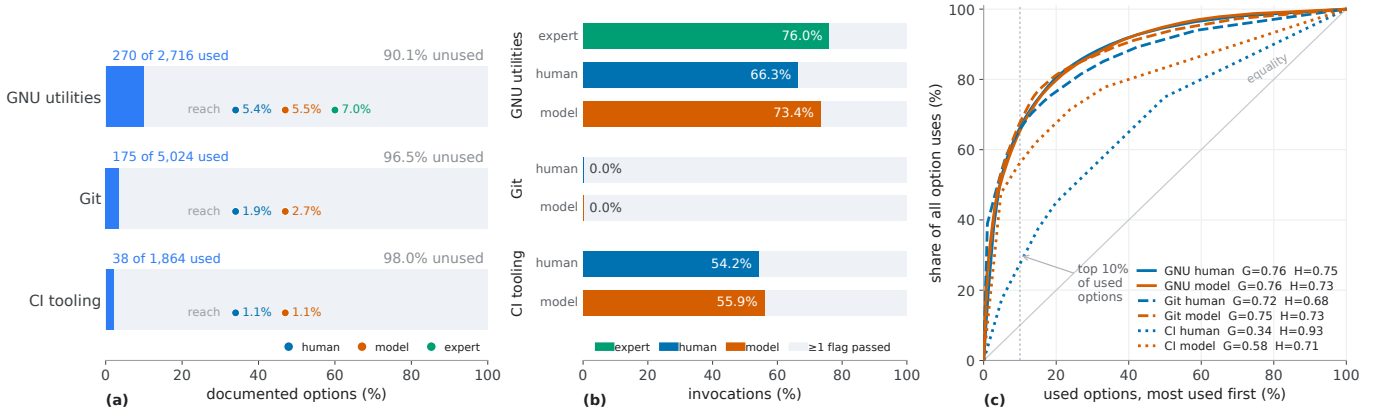


Fig. 1. Documented option saturation (*i.e.*, reach and concentration) of software programs across the three datasets and all three cohorts.

argument (*i.e.*, everything else). We record, for each invocation, the file it came from, the unit, and the set of options supplied. An invocation that supplies no options is retained as an explicit zero option record, which allows us to compute both the count of invocations and the distribution of options per invocation from the same table.

a) Handling aliases: Many options exist in two equivalent spellings, a short form and a long form, *e.g.*, `-r` and `--recursive` in `cp`. We identify alias pairs from the help output, where both forms appear on the same line, and record them. In the reach and partition analyses (*i.e.*, for answering RQ1 and RQ2) we count each spelling as a distinct option, because the caller’s choice of form is itself informative. In the style analysis (*i.e.*, for answering RQ3) we explicitly compare spelling preferences across cohorts. Moreover, it should be noted that the option names that are lexically identical across different commands are treated as distinct, for example, `ls -l` and `wc -l` are counted separately.

Temporal boundary. To ensure a strict separation between human- and model-generated artifacts in our study, we selected the GNU and kernel human/expert cohorts from commits in their respective repositories prior to January 2020, and the CI human cohort prior to the public release of ChatGPT (November 2022). The former precedes GPT-3 (June 2020) and the latter predates the first widely used code generating chat language models, minimizing the risk of language model influence. As for the artifacts in the large language model cohort, we identified them through explicit provenance markers, such as commit messages, file headers, or README attributions, that directly name the generating model.

The analysis pipeline is deterministic with Bash scripts.

F. Metrics

Each research question is assessed using one or more metrics, computed consistently across the human baseline, the model cohort, and, for Dataset A, the expert reference, ensuring direct comparability.

Reach (RQ1, RQ2): Reach measures how much of the documented option set a cohort actually uses. Let D denote

the set of options documented across all units (*i.e.*, in the ground truth), and $U \subseteq D$ the set of distinct options used by a given cohort. The reach is defined as the ratio

$$\rho = |U|/|D|,$$

so that the fraction $1 - \rho$ shows the set of untouched documented options by the cohort.

Concentration (RQ1): Reach measures how many options a cohort uses, but not how evenly usage is distributed across them. Concentration captures this imbalance. Let $x_1, \dots, x_k$ be the usage counts of the k options a cohort uses, with mean $\bar{x}$ and shares $p_i = x_i / \sum_j x_j$. We report two complementary measures. The Gini coefficient [25],

$$G = \frac{\sum_i \sum_j |x_i - x_j|}{2k^2 \bar{x}} \in [0, 1],$$

which equals 0 when all used options are called equally often and approaches 1 when usage piles onto a single option. And, the complementary normalised Shannon entropy [26],

$$H = -\frac{1}{\ln k} \sum_i p_i \ln p_i \in [0, 1],$$

which reads the same imbalance inversely, falling towards 0 as usage concentrates on fewer options.

Partition (RQ2): Concentration describes a single cohort. Partition compares the human baseline and the model cohort by splitting the options they use into mutually exclusive subsets, namely those used by both (*shared*), only by the human cohort (*human only*), only by the model cohort (*model only*), and by neither (*never used*). For Dataset A we additionally mark which of the never used options the expert cohort reaches, placing both footprints against an expert frontier.

Style (RQ3): The preceding metrics record *which* options a cohort uses. Style asks, for the options the human and model cohorts share, whether they are used *alike*, along three facets. *Popularity* is the Spearman rank correlation [27] between the two cohorts’ per-option usage counts. *Spelling* is the share of uses written in the short rather than the long form, restricted to the alias pairs of Section III-E so that a genuine choice

between forms exists. *Configuration depth* is the distribution of the number of options used per invocation.

IV. RESULTS

In this section, we structure findings around our three research questions and close with an interpretation.

A. Most of the Option Space is Never Used

Figure 1(a) shows, for each dataset, the share of documented options that any caller actually uses. The first thing to notice is how little of the options space is ever touched. For the GNU programs only 270 of 2,716 options ever appear in a script, leaving 90.1% completely untouched. Git and CI tooling are even more extreme, at 96.5% and 98.0% unused respectively. Across the three datasets that differ in domain, authorship, and structure, the finding is the same: about nine in ten documented options go unused by anyone. Even the most exploratory cohort, the Linux kernel scripts, *reaches* only 7.0% of GNU options, while models and humans both stay around 5.4% to 5.5% for GNU and below 3% for Git and CI.

One might expect this to change with more usage data, but the scripts we analyse already contain over 31,000 invocations across all cohorts, so the idle region is not a result of a thin sample. What the data shows instead is that calls are both shallow and concentrated. Specifically, as Figure 1b) shows, between 66.3% and 76.0% of GNU program invocations pass no option at all, meaning the program is simply called with its defaults. When options are used, they pile onto a small set of popular options. To quantify this *concentration*, we measure the Gini coefficient, a standard metric of inequality [25]. As a result, Figure 1c) shows that its value reaches 0.76 for GNU, meaning the top tenth of used options accounts for 66% to 68% of all calls, while the remaining nine tenths are almost never seen. The normalised Shannon entropy confirms this imbalance, where $H = 0.75$ for humans and $H = 0.73$ for models on GNU, well below the uniform maximum of 1.0. The mean sits at 0.45 options per human call and 0.34 per model call. The one exception is the human CI cohort, which spreads more evenly (Gini 0.34, $H = 0.93$), but even there the total slice of the option space reached is just as narrow.

These data highlight two important points. First that software programs accumulate options over decades for edge cases and portability, while any single script uses only the few its task requires. Then, the current language models seem to land on the same narrow slice that human scripts have always used.

Finding 1. Across all three domains, roughly 90% of documented options of programs are never exercised by any cohort. Usage is concentrated on a small core (Gini $G \geq 0.75$, Shannon $H \leq 0.75$), and two thirds of program invocations pass zero options. Importantly, models operate within this same narrow band and do not explore further.

B. Models Reach Like the Average Human, Not the Expert

Figure 2 shows how far into the documented option space each cohort reaches. The first thing to notice is that models and

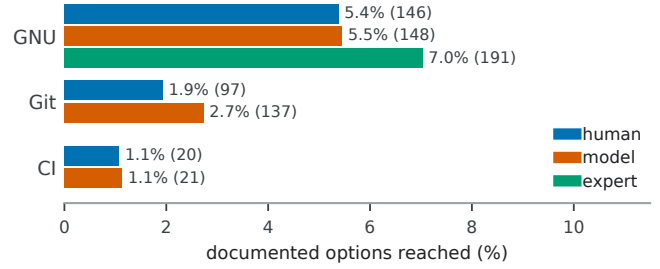


Fig. 2. Reach of documented program options by each cohort.

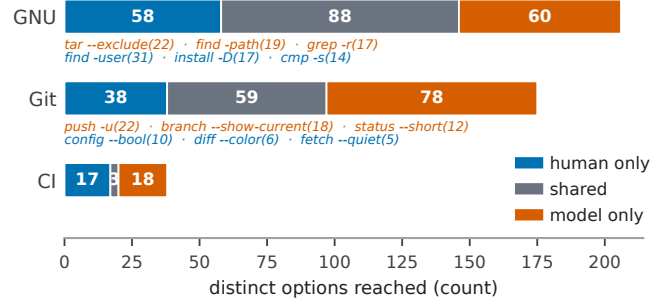


Fig. 3. Shared core and exclusive remainders of program options.

humans land at almost the same point. On the GNU programs the model cohort uses 5.45% of the options (148), and the human cohort uses 5.38% (146), a difference smaller than two options. The one cohort that genuinely stands apart is the expert kernel cohort at 7.03% (191 options). In other words, if there is a gap to pay attention to, it is between models and expert developers, not between models and average humans. What makes the expert cohort stand out is also worth noting. It is actually the most conservative caller, passing no option at all in 76% of invocations (*cf.* Figure 1b)), yet it still reaches the widest slice of the option space by drawing on a larger variety of distinct options whenever it does use one.

The other two datasets show the same result, with one exception. For CI tooling the two cohorts are virtually tied, with models reaching 1.13% (21 options) and humans 1.07% (20 options), which makes sense because both rely on the same few options for Docker, npm, and pip programs. For Git, however, models reach noticeably further, 2.73% (137 options) against 1.93% (97 options) for humans. This is the one dataset where models consistently do more with the program options, since every Git invocation in both cohorts carries at least one option and models simply draw from a wider set of them.

The next question that we asked is whether the options models and humans use are even the same ones. Figure 3 breaks the used options into a shared core and two exclusive portions. For the GNU programs, 88 options appear on both sides, yet 58 are used only by humans and 60 only by models, meaning each cohort regularly uses options the other never touches. On Git the exclusive model portion grows to 78

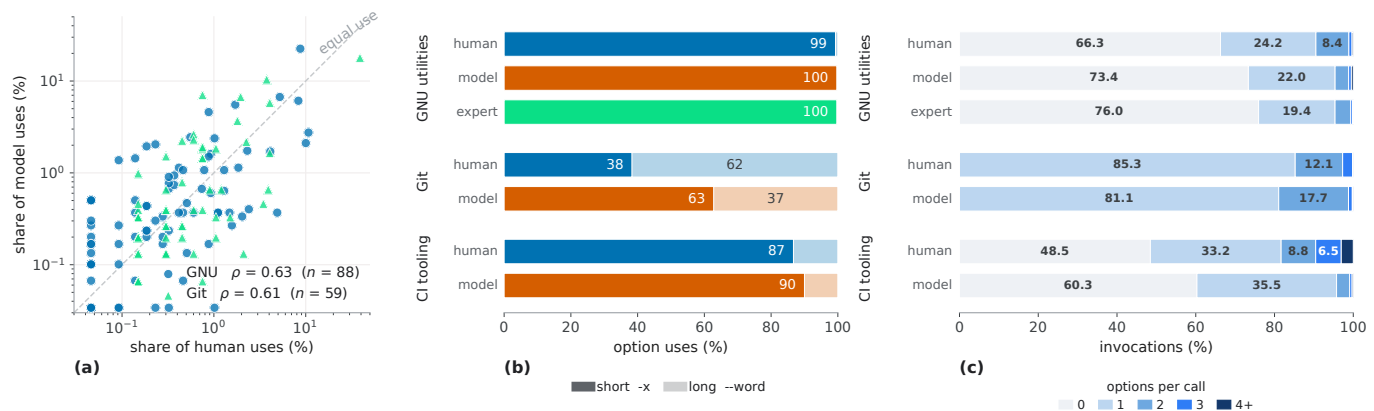


Fig. 4. Style agreement of models and humans across shared options: (a) popularity, (b) spelling form, and (c) configuration depth.

while the human portion stays at 38, again consistent with models covering more ground. The Figure 3 shows the top options on each side, and we noticed a consistent pattern across datasets. Namely, options that only models use tend to be general purpose, such as excluding files from an archive or searching recursively across a directory tree. Options that only humans use tend to be tied to a specific task, such as looking up file ownership, installing a file with explicit permissions, or comparing two files silently inside a conditional.

These data show that models cover the options that any user would reach for in a general task, but they miss or less use the ones that show up only when a script has a more specific task to do, such as checking ownership before installing a file, or comparing two binaries as part of a build check.

Finding 2. Model reach matches the typical human (5.45% vs. 5.38% on GNU) and diverges only from experts (7.03%). Each cohort holds an exclusive set of options the other never uses, of comparable size. Git is the sole exception where models reach wider than humans.

C. Same Popularity, Same Spelling, Different Depth

Regarding the shared options, the first thing to note is that models rank them almost exactly as humans do. Figure 4a) plots each shared option at its share of human uses against its share of model uses, and the points trend along the diagonal. The Spearman rank correlation is 0.63 over 88 shared GNU options ($p < 10^{-9}$) and 0.61 over 59 shared Git options, both moderate and statistically significant. The CI tooling is excluded because its shared core of only 3 options is too small for a meaningful rank test. Based on these two datasets, the options humans use most are also the ones models use most.

This agreement extends to *spelling*. Figure 4b) restricts attention to options that have both a short and a long form, so that a genuine choice exists. The results on the GNU programs show that both cohorts almost always pick the short form (e.g., `ls -l`), humans at 99% and models at 100%. On Git subcommands humans lean long (62%, e.g., `git commit --amend`) while models lean short (63%), and on CI tooling

both prefer the short form at 87% and 90% respectively. Across all aliased options, both cohorts agree on the majority form in 81 of 88 cases, with the few disagreements falling on Git. The divergence between cohorts is therefore about which options they use, not about how a shared option is spelled.

The one dimension where this agreement breaks down is how many options the two cohorts combine in a single call, i.e., the *depth of program configuration*. As Figure 4c) shows, we observed that the configuration of a program is shallow in both cohorts, with most invocations carrying zero or one option. This analysis covers all program invocations in each cohort, not only the programs shared between both. On the GNU programs the difference is modest, 9.5% of human program invocations carrying two or more options against 4.6% for models, and Git is similarly close at 14.7% against 18.9%. The CI tooling is the clear exception. There, 18.3% of human invocations carry two or more options and 3.1% carry four or more, against 4.2% and 0.3% for models.

This is not a reach deficit, since RQ2 showed the two cohorts reach almost the same number of CI program options. Models instead spread those options across separate invocations rather than combining them into one, suggesting a difference in how commands are composed, i.e., configured by models and humans, rather than in which options are known.

Finding 3. On shared options between humans and models, popularity rankings correlate positively (Spearman $\rho \approx 0.6$) and spelling agrees in 81/88 aliased cases. Configuration depth is comparable on GNU and Git but diverges on CI, where 18.3% of human calls combine two or more options versus only 4.2% for models.

D. Synthesis: Models Lack Both Visibility and Task Context

What do these results mean in practice? A language model generating a program invocation today will select from its documented option and express it in the same form a human developer would. It will also favour the same popular subset of options in the same order. In this sense, models already handle routine program configuration well. Yet the option space of a

mature program is large because the tasks it serves are diverse, and most of that diversity stays untouched.

The gap has two sides. First, a model never consults the program’s option set before generating a command. A human developer can invoke `--help` or read a manual page to see what exists. The model has no such step and draws only on options it has seen in training text. Second, even if the options were visible, selecting the right one requires understanding what the task demands. An expert developer who needs to keep an archive on a single filesystem, verify it after writing, or suppress a comparison inside a conditional will select the one option that fits. That option is documented and available. But reaching it requires both knowing it exists and knowing the task calls for it. Our data indicate that this combination is what separates the expert from both the model and the average human. The same mechanism accounts for the depth gap on CI tooling. There, human scripts compose several options into one call to express a compound requirement, while models issue separate simpler calls.

The implication is that the limitation is not one of capability. A model can employ any option it encounters. What it lacks is visibility of the full option set and sufficient task context to select from it. Augmenting the prompt with the documented options of the invoked program addresses the first deficit directly. Encoding the task requirements addresses the second. Together, these represent the most actionable improvements our findings point to.

V. DISCUSSION: FROM DIAGNOSIS TO INTERVENTION

We ask whether executing agents widen the narrow reach of programs’ option space we measured in static artifacts, and propose making configuration spaces machine queryable.

A. Agent Executed Commands

Our corpora in Table I capture commands that models write into static artifacts within a codebase. A natural objection is that AI agents, which execute commands, observe their output, and adapt their behavior accordingly, might behave differently. So, we thought that, perhaps the ability to see what a command actually does prompts an agent to try options it would not otherwise have reached for. To test this we mined two publicly released trajectory corpora: 600 SWE-agent runs backed by Llama 70B [17] and 600 DeepSWE-Agent runs backed by Kimi K2 [28], extracting every GNU command each agent actually executed in its sandbox. The two corpora yielded 1,779 and 14,369 program invocations, respectively, of the programs that we covered by Dataset A.

Figure 5 shows the result. It can be noted that rather than widening reach, the feedback loop narrows it in both agents. Llama 70B touches only 17 distinct documented options, which is 0.6% of the GNU documented space, less than a seventh of what the static model cohort reaches. Kimi K2, a more capable model with far more invocations, reaches 36 options (1.3%), still under a quarter of human reach. Figure 5b) shows the divergence within this narrow set. Llama 70B concentrates on directory listing: `ls -F` accounts for 55% of its option

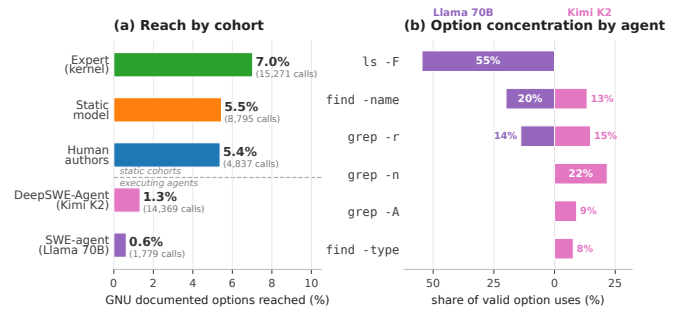


Fig. 5. Agent study comparing two executing agents, each on 600 trajectories.

uses. Kimi K2 concentrates instead on code search: `grep -n`, `grep -r`, and `find -name` together account for just under 50% of its uses. The two profiles are agent specific, but equally narrow. Observing output confirms what a command did, but gives no information about the options that were never tried.

Another important thing is that validity of the executed commands remains high for both agents, at 99.4% for Llama 70B and 95.8% for Kimi K2. The lower rate for Kimi K2 partly reflects its aggressive use of multi-step pipelines, where option attribution to the outer command is conservative. In both cases, the few invalid options carry a cost that static artifacts do not, *i.e.*, in a static script, an invalid option is a latent bug, whereas in an agent loop, it is a spent action step. The study spans two agent families and one task distribution, not a fourth dataset, but it removes any doubt that the narrow reach measured in static artifacts is an artifact of static authorship. When a model executes rather than writes, reach narrows further, and the cost of the missing options is paid immediately.

B. Proposal for Machine Queryable Configuration Spaces

Programs expose configuration space through interfaces designed for humans. A developer reads `--help` and selects the right option. By default, a model does not do this at generation time, and help text omits what correct selection requires. Option dependencies, mutual exclusions, deprecation status, and security advisories live in manuals, changelogs, or community knowledge beyond the training cutoff. Our first results show that making a model execute commands does not widen its reach (Figure 5), so the gap is not one of effort but of *visibility*. We propose a two layer intervention, *configuration manifests* to make the option space of software programs visible, and *configuration contracts* to keep that visibility accurate across versions and implementations.

Layer 1, Configuration Manifests. Ideally, programs would publish a machine readable *configuration manifest* alongside their human readable help, so that the full configuration space becomes visible to model callers at generation time. For instance, a flag such as `--llm` would emit a structured declaration of each option with its type, valid values, conflicts, and co-requirements. A model receiving this manifest before generating a command would know which combinations are valid and which carry security issues. This would make the

program self describing, analogous to a web API with an OpenAPI specification. Producing such a manifest would not require the program to be installed locally. Its configuration space can instead be extracted statically from argument parser definitions in source code, or published as a standalone artifact in repositories and package registries.

The Model Context Protocol (MCP) [29] already standardises how models discover external tools by querying a registry for schemas at runtime. If programs exposed such a manifest, they would become first class MCP tools whose option schemas are queryable through the same protocol models already use for function calling. The infrastructure for tool discovery exists, but programs do not yet advertise their configuration space in a form it can serve.

Adopting manifests would also pressure bloated programs that have accumulated hundreds of corner case options [30] to factor them into smaller specialised units, reducing functionality and interface complexity for both humans and models.

Layer 2, Configuration Contracts. A manifest solves the visibility at a single point in time, but options are added, deprecated, and removed between versions. A model trained on text from 2023 cannot know that an option was deprecated in 2025, and no amount of training or prompting prevents this class of error. Nor does a manifest resolve *identity*. A model encountering `ls` does not know whether the target is GNU coreutils, BusyBox, or Toybox, yet they expose different option sets. Generating `--sort=version` for BusyBox fails silently. These two gaps, temporal evolution and implementation ambiguity, fall outside what a static manifest can provide.

We therefore propose that programs additionally adopt *configuration contracts*, analogous to API contracts in service oriented architectures. A contract is machine parseable, version stamped, declares implementation identity, and encodes inter option constraints that help text leaves implicit. Where a manifest answers “what options exist right now?”, a contract answers “what changed since the version you know, and which binary are you actually talking to?” This mechanism generalises beyond the command line to compile time feature flags, environment variables, and configuration files, all of which suffer from the same visibility gap our data documents.

Together, *manifests and contracts shift the burden from training to infrastructure*. Rather than hoping a model has seen enough examples, the program tells the model what it supports right now. Making the full configuration space of programs visible, versioned, and queryable to models at generation time is the architectural intervention our findings call for.

VI. THREATS TO VALIDITY

Construct Validity: We recover each program’s option set from its installed help output, pinned to a specific version. A program may have undocumented options, making our denominator too small and our reach numbers upper bounds. Since the main finding is that reach is small, this makes the conclusion stronger, not weaker. Then, regarding whether all identified options in program invocations are correctly attributed, our reach computation counts an extracted option

only when it matches an entry in the program’s ground truth, so the invalid options do not inflate the reported numbers.

We identify model scripts by explicit provenance markers in commits, headers, or READMEs. Any unmarked model script lands in the human cohort, compressing the observed gap rather than inflating it. Moreover, we treat all model generated scripts as a single cohort because the combined reach is already so narrow that splitting it by generating model (ChatGPT, Claude, GPT) would not alter the finding.

Internal Validity: The human and model corpora come from different repositories, so differences may partly reflect different tasks rather than different callers. We mitigate this with large and diverse corpora and by checking that findings replicate across three unrelated datasets. The Git result, where models reach wider than humans, confirms that our chosen methodology does not mechanically favour one side.

The kernel expert cohort is a single codebase, so its 7.03% reach is a reference point, not a population estimate. The general human cohort also draws from mature projects whose contributors may be equally skilled, so our human versus expert separation reflects codebase provenance rather than verified expertise. The agent study covers one task distribution. A broader task mix might produce wider reach.

External Validity: The central finding holds across three datasets that differ in domain, tool family, and artifact structure. Still, other tool families may behave differently, and the precise numbers should be read as characteristic of these corpora, not as universal constants. The model cohort is a snapshot of ChatGPT, Claude, and GPT variants. Newer models may reach further. The two agents cover a small slice of the agent design space, and extending to other architectures is a direct next step.

VII. RELATED WORK

Configuration in Software Engineering: A large body of work studies software configuration as a source of complexity and failure. Xu *et.al.*, showed that configuration spaces keep growing while most options are set by very few users [4]. Surveys tied this unused space to real production incidents [16], and field studies treated configuration as an engineering discipline in its own right [3]. Others localise the option behind a given behaviour [31], model how options affect performance [12], and mine the constraints that make a configuration valid [13], which a recent review frames as learning the structure of a configuration space from observed usage [32]. We share this framing but ask a different question. This literature studies human use of configuration, whereas our subject is the model. We measure how far a model reaches into the option space of the programs it calls, and read human usage only as the reference baseline that tells us whether a narrow model reach is unusual or simply how these programs are used.

Variability and Product Lines: The software product line community formalised a configuration space as a structured set of features with constraints among them [1], [2], [33]. Work in that community measured how much of a configurable space

tests and users actually exercise [34], [35], and showed how far the space a system advertises can drift from the space its artifacts use [36]. We take from this the idea that the documented space and the used space are separate objects, and we measure the gap for the command line programs a model calls, with a programs’s help output as the advertised space and the model’s mined invocations as the used one.

Evaluation of Model Generated Code: Most evaluation of code generation checks functional correctness against curated benchmarks [5], [37], [22] or real repository issues [23]. Other work studies how developers trust and use these tools [7], [8], and shows that generated code can carry vulnerabilities [10] or invent package names that never existed, mirroring the training distribution rather than reality [21]. These efforts ask whether generated code works or whether it is safe. We ask whether it uses the option space of the programs it calls, which a passing test never checks. The package hallucination result is the closest to ours in spirit, since both show a model falling back on a narrow training prior, but we measure that narrowing on options that are real and documented rather than on names that do not exist.

Tool Use and the Command Line: Work on tool augmented models studies how a model calls external functions and APIs [38], [39], [40]. These frameworks assume a fixed catalogue of tools described by a schema, now standardised in the Model Context Protocol [29], and study which tool to pick and how to fill its named arguments. Even then, models often pick the wrong tool when several fit [41], [42], [43]. The command line has no such schema. Its very identity shifts across hosts, so neither the implementation that will run nor the options it accepts are known to the model in advance. Recent agent systems hand the model a shell directly, so it emits raw invocations that run in a sandbox and feed their output back as observations [18], [44], [19], which has prompted emulated sandboxes and benchmarks for whether a model notices when an executed command is unsafe [45], [46]. The command line is also a target for natural language interfaces, from the NL2Bash corpus pairing English with shell commands [47] to later work on translation accuracy and judging when two commands are equivalent [48], and for agent benchmarks that score whether hard terminal tasks are solved [49]. All of this treats the command as the output to produce or the action to run. We instead treat the command as the object of study and ask how far a model reaches into the option space of each program it invokes, against human usage as the reference. To our knowledge, no prior work measures how a model uses the configuration spaces of the command line tools it calls.

VIII. CONCLUSION

In this paper, we measured how far large language models reach into the documented option spaces of command line programs. For this purpose, we analysed 403 programs across three domains invoked in various contexts, by human and model cohorts, over 31,000 times. In every case, we found out that roughly nine in ten program options go unused by

all cohorts alike. Models reach about as far as average human authors, and only the expert reference, drawn from the Linux kernel build system, reaches noticeably wider. Where a model and a human share a program option, they tend to rank it alike and spell it in the same form, so the difference lies in which options each cohort selects, not in how they handle the ones they select. Our agent study shows that executing commands and observing their output does not widen reach either.

These results show that models reproduce the frequency profile of their training text. The program options they never pick are the ones that were rare in that text, regardless of whether a task would benefit from them. The expert reaches wider not because of familiarity with more obscure options but because a concrete requirement points to a specific option. That kind of selection is driven by the task at hand, and it is precisely what a model cannot do when it composes an invocation without seeing what the program documents.

To close this gap, we propose that programs publish machine readable configuration manifests and version stamped configuration contracts, so that models can query the full option set at generation time rather than relying on training memory alone. Our dataset and metrics provide a baseline against which any such intervention can be evaluated.

Future Work: Our first experiment is to provide the model with the full set of program options at generation time and test whether reach changes. If it does, the follow-up question is how best to present that information, since a raw option list may exceed the prompt budget and a task-conditioned summary may work better. A second direction is to move beyond single options. We measured reach one option at a time, but the right choice often depends on neighbouring options and on other commands in a pipeline. Modelling which option combinations are valid and which a task actually needs would turn a count of reached options into a measure of whether a command was well configured. Thirdly, we know that broader reach of program options does not guarantee better outcomes. Pairing reach with measured results, such as reproducibility, robustness, or a passing build, would show which unused program options are worth surfacing for models. Finally, our agent study covers two systems and one task distribution. A full study across more agent families and more tasks would clarify whether and how execution based feedback can complement program documentation of options.

REFERENCES

- [1] S. Apel, D. Batory, C. Kästner, and G. Saake, *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, 2013. <https://doi.org/10.1007/978-3-642-37521-7>
- [2] T. Berger, R. Rublack, D. Nair, J. M. Atlee, M. Becker, K. Czarnecki, and A. Wasowski, “A survey of variability modeling in industrial practice,” in *Proceedings of the 7th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, 2013, pp. 7:1–7:8. <https://doi.org/10.1145/2430502.2430513>
- [3] M. Sayagh, N. Kerzazi, B. Adams, and F. Petrillo, “Software configuration engineering in practice: Interviews, survey, and systematic literature review,” *IEEE Transactions on Software Engineering*, vol. 46, no. 6, pp. 646–673, 2020. <https://doi.org/10.1109/TSE.2018.2867847>

- [4] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadder, "Hey, you have given me too many knobs! Understanding and dealing with over-designed configuration in system software," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2015, pp. 307–319. <https://doi.org/10.1145/2786805.2786852>
- [5] M. Chen, J. Twarek, H. Jun *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. <https://doi.org/10.48550/arXiv.2107.03374>
- [6] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. <https://arxiv.org/abs/2203.13474>
- [7] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM, 2022, pp. 332:1–332:7. <https://doi.org/10.1145/3491101.3519665>
- [8] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proceedings of the ACM on Programming Languages (PACMPL)*, vol. 7, no. OOPSLA1, pp. 85–111, 2023. <https://doi.org/10.1145/3586030>
- [9] Reproducible Builds Project, "Archive metadata: Creating reproducible tar archives," <https://reproducible-builds.org/docs/archives/>, 2024, accessed 2026.
- [10] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of github copilot's code contributions," in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>
- [11] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 426–437. <https://doi.org/10.1145/2970276.2970358>
- [12] N. Siegmund, A. Grebhorn, S. Apel, and C. Kästner, "Performance-influence models for highly configurable systems," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2015, pp. 284–294. <https://doi.org/10.1145/2786805.2786845>
- [13] S. Nadi, T. Berger, C. Kästner, and K. Czarnecki, "Mining configuration constraints: Static analyses and empirical results," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 140–151. <https://doi.org/10.1145/2568225.2568283>
- [14] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An empirical study on configuration errors in commercial and open source systems," in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2011, pp. 159–172. <https://doi.org/10.1145/2043556.2043572>
- [15] H. S. Gunawi, M. Hao, T. Leesatapornwongsa, T. Patana-anake, T. Do, J. Adityatama, K. J. Eliazar, A. Laksono, J. F. Lukman, V. Martin, and A. D. Satria, "What bugs live in the cloud? A study of 3000+ issues in cloud systems," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. ACM, 2014, pp. 7:1–7:14. <https://doi.org/10.1145/2670979.2670986>
- [16] T. Xu and Y. Zhou, "Systems approaches to tackling configuration errors: A survey," *ACM Computing Surveys*, vol. 47, no. 4, pp. 70:1–70:41, 2015. <https://doi.org/10.1145/2791577>
- [17] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. <https://arxiv.org/abs/2405.15793>
- [18] J. Yang, A. Prabhakar, K. Narasimhan, and S. Yao, "InterCode: Standardizing and benchmarking interactive coding with execution feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. <https://arxiv.org/abs/2306.14898>
- [19] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "OpenHands: An open platform for AI software developers as generalist agents," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. <https://arxiv.org/abs/2407.16741>
- [20] Anthropic, "Tool use with claude: the bash tool and command execution," Anthropic Developer Documentation, 2024, accessed 2026.
- [21] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadhwal, "We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security)*, 2025, pp. 3687–3706. <https://dl.acm.org/doi/10.5555/3766078.3766268>
- [22] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. <https://dl.acm.org/doi/10.5555/3666122.3667065>
- [23] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2310.06770>
- [24] V. R. Basili, G. Caldiera, and H. D. Rombach, "The goal question metric approach," in *Encyclopedia of Software Engineering*. John Wiley & Sons, 1994.
- [25] C. Gini, "Measurement of inequality of incomes," *The Economic Journal*, vol. 31, no. 121, pp. 124–126, 1921. <https://doi.org/10.2307/2223319>
- [26] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- [27] C. Spearman, "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904. <https://doi.org/10.2307/1412159>
- [28] SWE-Factory, "DeepSWE-Agent trajectories with Kimi K2," Hugging Face Dataset, <https://huggingface.co/datasets/SWE-Factory/DeepSWE-Agent-Kimi-K2-Trajectories-2.8K>, 2025.
- [29] Anthropic, "Introducing the Model Context Protocol," Anthropic Announcement, 2024, accessed 2026. <https://www.anthropic.com/news/model-context-protocol>
- [30] G. J. Holzmann, "Code inflation," *IEEE Software*, vol. 32, no. 2, pp. 10–13, 2015.
- [31] S. Zhang and M. D. Ernst, "Which configuration option should i change?" in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. ACM, 2014, pp. 152–163. <https://doi.org/10.1145/2568225.2568251>
- [32] J. A. Pereira, M. Acher, H. Martin, J.-M. Jézéquel, G. Botterweck, and A. Ventresque, "Learning software configuration spaces: A systematic literature review," *Journal of Systems and Software*, vol. 182, p. 111044, 2021. <https://doi.org/10.1016/j.jss.2021.111044>
- [33] C. Kästner, S. Apel, T. Thüm, and G. Saake, "Type checking annotation-based product lines," *ACM Transactions on Software Engineering and Methodology*, vol. 21, no. 3, pp. 14:1–14:39, 2012. <https://doi.org/10.1145/2211616.2211617>
- [34] A. Halin, A. Nuttinck, M. Acher, X. Devroey, G. Perrouin, and B. Baudry, "Test them all, is it worth it? Assessing configuration sampling on the jhipster web development stack," *Empirical Software Engineering*, vol. 24, no. 2, pp. 674–717, 2019. <https://doi.org/10.1007/s10664-018-9635-4>
- [35] D. Jin, X. Qu, M. B. Cohen, and B. Robinson, "Configurations everywhere: Implications for testing and debugging in practice," in *Companion Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 215–224. <https://doi.org/10.1145/2591062.2591191>
- [36] A. Knüppel, T. Thüm, S. Mennicke, J. Meinicke, and I. Schaefer, "Is there a mismatch between real-world feature models and product-line research?" in *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2017, pp. 291–302. <https://doi.org/10.1145/3106237.3106252>
- [37] J. Austin, A. Odena, M. Nye *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021. <https://doi.org/10.48550/arXiv.2108.07732>
- [38] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. <https://dl.acm.org/doi/10.5555/3666122.3669119>

- [39] Y. Qin, S. Liang, Y. Ye *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2307.16789>
- [40] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. <https://arxiv.org/pdf/2210.03629>
- [41] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive apis,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. <https://doi.org/10.52202/079017-4020>
- [42] Y. Qin, S. Hu, Y. Lin *et al.*, “Tool learning with foundation models,” *ACM Computing Surveys*, vol. 57, no. 4, pp. 101:1–101:40, 2025. <https://doi.org/10.1145/3704435>
- [43] Y. Huang, J. Shi, Y. Li, C. Fan, S. Wu, Q. Zhang, Y. Liu, P. Zhou, Y. Wan, N. Z. Gong, and L. Sun, “MetaTool benchmark for large language models: Deciding whether to use tools and which to use,” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2310.03128>
- [44] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework,” in *Proceedings of the Conference on Language Modeling (COLM)*, 2024. <https://arxiv.org/abs/2308.08155>
- [45] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2309.15817>
- [46] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, “R-Judge: Benchmarking safety risk awareness for LLM agents,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2024, pp. 1467–1490. <https://doi.org/10.18653/v1/2024.findings-emnlp.79>
- [47] X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst, “NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system,” in *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC)*, 2018. <https://arxiv.org/abs/1802.08979>
- [48] F. Westenfelder, E. Hemberg, S. Moskal, U.-M. O’Reilly, and S. Chiricescu, “LLM-supported natural language to Bash translation,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2025, pp. 11 135–11 147. <https://doi.org/10.18653/v1/2025.naacl-long.555>
- [49] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj *et al.*, “Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces,” *arXiv preprint arXiv:2601.11868*, 2026. <https://doi.org/10.48550/arXiv.2601.11868>